

UNIVERSITY OF ESWATINI

Department of Computer Science

Software Requirements Specification

Smart Gate Access Control System

Date: 13 May 2026

Prepared by:

Nomsa Manana · Celimpilo Mdluli · Sizolwethu Dlamini · Nomkhosi Myeni

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 User Characteristics
 - 2.3 Operating Environment
 - 2.3.1 Gate Workstations
 - 2.3.2 Database Server
 - 2.3.3 Development and Build Environment
 - 2.4 Design and Implementation Constraints (System Business Rules)
 - 2.5 Assumptions and Dependencies
 - 2.5.1 Assumptions
 - 2.5.2 Dependencies
 3. Functional Requirements with Acceptance Criteria
 - Functional Requirements (FR)
 - Non-Functional Requirements (NFR)
 4. Interface Requirements (sic)
 - 5.1 User Interfaces
 - 5.1.1 Login Screen
 - 5.1.2 Gate Activity Panel (Guard View)
 - 5.1.3 Administrator Dashboard
 - 5.1.4 Visual Design Constraints
 - 5.2 Hardware Interfaces
 - 5.3 Software Interfaces
 - 5.4 Communication Interfaces
 5. Requirements Traceability Matrix
- Appendices
- Appendix A: Database Schema
 - Appendix B: Acceptance Criteria Summary
 - Appendix C: Glossary

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete set of functional and non-functional requirements for the Smart Gate Access Control System, developed for the University of Eswatini (UNESWA). The document is prepared in accordance with IEEE Std 830-1998 and is intended for the following audiences:

Person	Purpose
Development team	As the authoritative specification for design and implementation decisions.
Quality assurance and testing personnel	As the basis for test case derivation and acceptance criteria.
University administration.	To confirm that the system meets operational and institutional requirements.
Academic supervisor	For progress assessment and compliance verification.
Future maintainers	As the foundation for understanding, modifying, or extending the system.

1.2 Scope

The Smart Gate Access Control System is a Java SE desktop application integrated with a centralized MySQL 8.x database. It replaces manual logbooks at the University of Eswatini campus gates with an automated, card-based entry and exit management system.

In Scope - The system provides:

- Barcode and QR code card scanning via the ZXing library, with manual ID fallback (for simulation purposes).
- Real-time gate activity logging (user identity, action, gate point, timestamp).
- An administrative dashboard with user management, live statistics, and report generation.
- Role-Based Access Control (RBAC) distinguishing ADMIN, STAFF, and STUDENT roles.
- Offline fallback mode with local caching and automatic re-synchronisation.

Out of Scope - The following are explicitly excluded from the current scope:

- Biometric authentication (fingerprint or facial recognition).
- Mobile application interfaces.

- Off-campus or satellite campus access points.
- Real-time two-way integration with external HR or student records systems beyond an initial one-time data import.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
UNESWA	University of Eswatini – the client and deployment site for this system.
UID	Unique identifier stored on a university-issued barcode card.
RBAC	Role-Based Access Control – restricts system functions based on the authenticated user's role.
JDBC	Java Database Connectivity – standard Java API for relational database access.
ZXing	Zebra Crossing – open-source Java library for decoding 1D and 2D barcodes.
Gate Activity	A log record comprising user ID, full name, action (ENTRY/EXIT), gate identifier, and timestamp.
Fallback Mode	Operational mode activated when database connectivity is unavailable; guards enter IDs manually and events are cached locally.
UAT	User Acceptance Testing – structured testing with representative end users.
SUS	System Usability Scale – a standardised 10-item questionnaire yielding a score from 0 to 100.
MVC	Model-View-Controller – architectural pattern separating data, interface, and logic.
3NF	Third Normal Form – a database normalisation level that eliminates transitive dependencies.
HID	Human Interface Device – USB device class used by barcode scanners to emulate keyboard input.
SRS	Software Requirements Specification – this document.
DXA	Document eXtension unit – 1/1440 of an inch, used in Word/OOXML formatting.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998.
- Smart Gate Project Plan, University of Eswatini, Department of Computer Science, 2025.
- Oracle Corporation, Java Code Conventions, 2020 revision.
- ISO/IEC 25010:2021, Systems and Software Quality Models. ISO/IEC, 2021.
- ZXing Project, ZXing Core Library Documentation. GitHub, 2023.
- MySQL 8.0 Reference Manual. Oracle Corporation, 2023.

1.5 Overview

This SRS is organized into seven sections.

Section 1 (**Introduction**) establishes scope, definitions, and references.

Section 2 (**Overall Description**) provides product context, user characteristics, operating environment, and constraints.

Section 3 (**Functional Requirements**) enumerates all system functions with acceptance criteria.

Section 4 (**Non-Functional Requirements**) specifies measurable quality attributes.

Section 5 (**Interface Requirements**) details user, hardware, software, and communication interfaces.

Section 6 (**Requirements Traceability Matrix**) maps each requirement to project objectives. Appendices provide the database schema, acceptance criteria, and glossary.

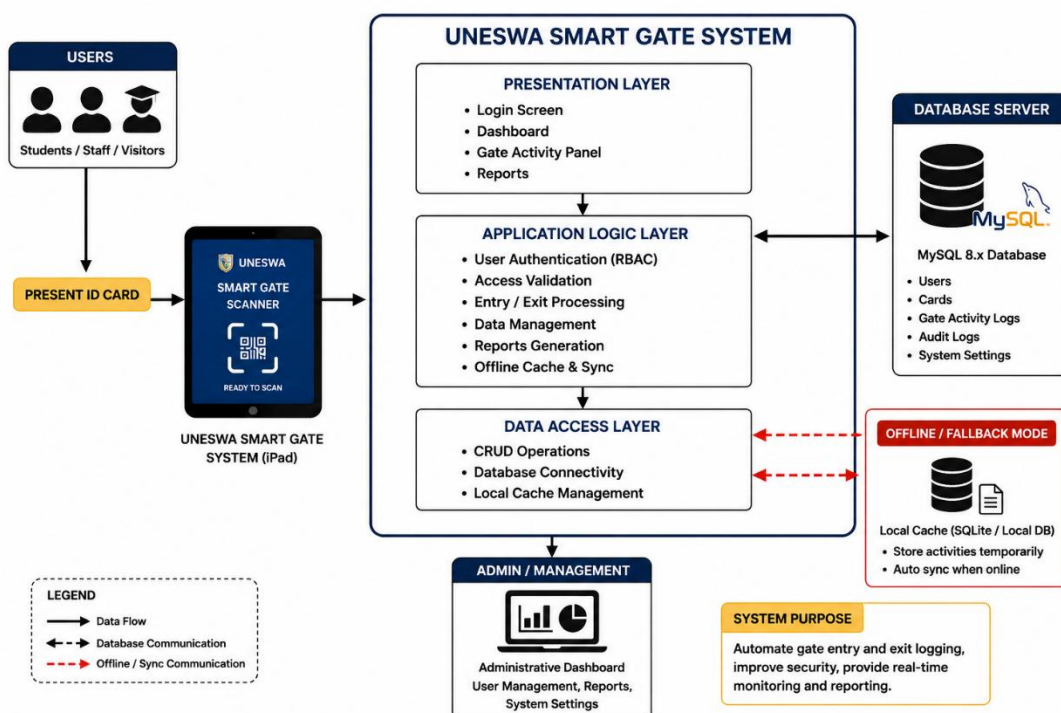
2. Overall Description

2.1 Product Perspective

The Smart Gate Access Control System is a standalone desktop application. It does not replace any existing third-party system but introduces automated digital access management where manual logbooks previously existed. It interfaces with the following external components:

- **USB barcode scanners:** provide card UID input to the gate client application via HID keyboard emulation.
- **MySQL 8.x database server:** centralised persistent storage for users, card associations, gate activity logs, and security audit records.
- **University identity data:** a one-time import of student and staff records during initial deployment; no ongoing real-time integration.

A conceptual block diagram of the system's layered architecture is described below. The system employs a three-tier model: (1) Presentation Layer — Java Swing GUI components (LoginFrame, DashboardFrame, GateActivityPanel); (2) Business Logic Layer — RBAC enforcement, session management, access-rule evaluation; (3) Data Access Layer — JDBC PreparedStatement-based interaction with the MySQL server.



2.2 User Characteristics

User Role	Technical Proficiency	Primary Responsibilities
Security Guard	Basic computer literacy	Card scanning, manual ID entry, viewing current gate activity, activating fallback mode.
Administrator	Moderate (familiar with web dashboards)	User registration, report generation, dashboard monitoring, system configuration, RBAC management.
Student / Staff	Minimal (card-holder only)	Presenting ID card at gate scanner; no direct system interaction beyond card swipe.
IT Maintainer	Advanced (database and server administration)	Database backups, server maintenance, log auditing, deployment, connection configuration.

2.3 Operating Environment

2.3.1 Gate Workstations

- Operating system: Windows 10 or Windows 11 (64-bit).
- Runtime: Java Runtime Environment (JRE) 11 or later.
- Hardware: At least one USB port for the barcode scanner; minimum 4 GB RAM; 1280 × 720 display resolution.

2.3.2 Database Server

- Operating system: Ubuntu Server 22.04 LTS.
- Database: MySQL 8.0.x (InnoDB engine) with MySQL Connector/J 8.0.x.
- Runtime: Java Development Kit (JDK) 11 or later.

2.3.3 Development and Build Environment

- IDE: IntelliJ IDEA Community Edition.
- Build tool: Apache Maven 3.9.x.
- Version control: Git / GitHub.

2.4 Design and Implementation Constraints (System Business Rules)

- The system shall be developed exclusively in Java SE (JDK 11 or later) to ensure platform independence and alignment with the university's existing infrastructure.
- The graphical user interface shall be implemented using Java Swing; no third-party UI frameworks (such as JavaFX, Electron) are permitted.
- The database shall be MySQL 8.x using the InnoDB storage engine to guarantee ACID transaction support.
- All database interactions shall use JDBC PreparedStatement objects; dynamic string concatenation for SQL query construction is strictly prohibited.
- Passwords shall be stored as salted SHA-256 hashes; plain-text passwords shall never appear in logs, network transmissions, or persistent storage.
- The system shall provide a functional offline fallback mode when database connectivity is unavailable, caching events locally for subsequent synchronisation.
- Source code shall be managed in a Git repository hosted on GitHub; no direct commits to the main branch are permitted.
- All Java code shall conform to Oracle Java Code Conventions (2020 revision), including Javadoc on all public methods and classes.
- The MySQL schema shall comply with Third Normal Form (3NF) with foreign key constraints enforced at the database level.

2.5 Assumptions and Dependencies

2.5.1 Assumptions

- The University of Eswatini provides stable electrical power and network connectivity at all designated gate points throughout operating hours (07:00–22:00).
- All registered students and staff carry their university-issued barcode ID cards when accessing or exiting campus.
- Gate security guards have received a minimum of basic computer literacy training prior to system deployment.
- The university administration supplies anonymized sample student and staff data for development-phase testing.

2.5.2 Dependencies

- Availability and procurement of ZXing-compatible USB barcode scanners at each designated gate workstation.
- The IT Department provisions and maintains a MySQL 8.x server accessible from all gate client workstations via TCP/IP on port 3306.
- The IT Department ensures JDBC connectivity and configures any required firewall rules or VPN access for the gate-to-server communication.

3. Functional Requirements with Acceptance Criteria

Summarized below are all functional (FR) and Non-Functional (NFR) requirements are stated from an externally observable perspective, consistent with IEEE Std 830-1998 §5.3.2. Each requirement is assigned a unique identifier, a descriptive title, a precise statement, and a measurable acceptance criterion.

Functional Requirements (FR)

FR-01 : Card Scanning	Scans barcode/QR cards and retrieves user information.
Acceptance: User details must display within 2 seconds after scan.	
FR-02 : Real-Time Activity Logging	Automatically records gate activities in the database.
Acceptance: New records must appear in dashboard feed within 5 seconds.	
FR-03 : Manual Fallback Entry	Allows manual ID entry when scanner fails.
Acceptance: Valid IDs must log successfully within 3 seconds.	
FR-04 : Campus Statistics Summary	Displays live entry, exit, and campus statistics.
Acceptance: Dashboard values must refresh every 30 seconds.	
FR-05 : User Registration	Admins can register users and assign cards.
Acceptance: Duplicate IDs and cards must be rejected.	
FR-06 : Report Generation	Generates reports and exports them to CSV.
Acceptance: Reports up to 10,000 rows must load within 60 seconds.	

Non-Functional Requirements (NFR)

NFR-01 : Scan Response Time	System must respond quickly during card scans.
Acceptance: Response time should not exceed 2 seconds.	
NFR-02 : Concurrent Workstations	Supports multiple gates operating together.
Acceptance: At least 5 workstations must run simultaneously.	
NFR-03 : Password Security	Passwords must be securely encrypted.
Acceptance: No plain-text passwords should exist in the system.	
NFR-04 : Session Timeout	Inactive sessions should automatically expire.
Acceptance: Sessions must end after 30 minutes of inactivity.	
NFR-05 : System Availability	System should remain reliable during operation.
Acceptance: Availability target must be at least 99% uptime.	
NFR-06 : MVC Architecture	Application must follow MVC design structure.
Acceptance: All modules should separate View, Controller, and Model.	

5. Interface Requirements

5.1 User Interfaces

5.1.1 Login Screen

- Fields: username (ID number) text field, password field with masked input.
- Controls: Login button, error label for authentication failure messages.
- Behavior: Pressing Enter in either field submits the form; failed authentication increments a counter logged in security_log.

5.1.2 Gate Activity Panel (Guard View)

- Prominent scan status area displaying the last scanned user's name, ID, and action type (ENTRY/EXIT) in large, high-contrast text.
- Manual ID input field with a Submit button, visible when the scanner is unavailable.
- Scrollable table showing today's gate activity for the current gate point: timestamp, user name, ID, action.
- Persistent fallback-mode indicator banner when operating offline (NFR-13).

5.1.3 Administrator Dashboard

- Collapsible sidebar navigation with icons and labels for all five functional panels (FR-07).
- Statistics cards on the dashboard summary: total entries, total exits, on campus now, unique users today (FR-04).
- Real-time activity feed with auto-refresh every 30 seconds.
- User management table with CRUD controls: register, edit (name, email, role), deactivate (soft delete — no physical deletion).
- Report builder with date-range picker, gate selector, and role filter (FR-06); Export to CSV button.
- Settings panel for database connection parameters and institutional display name.

5.1.4 Visual Design Constraints

- All UI components shall adhere to the centralised Theme class: dark navy (#1B2A4A) and purple (#5C2D91) colour scheme, consistent font family (Arial or system sans-serif), and minimum 12pt body text.
- Colour contrast ratios shall meet WCAG 2.1 Level AA (minimum 4.5:1 for normal text) to support accessibility.

5.2 Hardware Interfaces

- Barcode scanners shall be USB Human Interface Device (HID) scanners compatible with ZXing Core 3.5.x, emulating keyboard input so that no custom device driver is required.
- Each gate workstation shall provide at least one available USB-A or USB-C port for the scanner.
- Scanner decode events are delivered to the application as keyboard input strings terminated by a carriage return (Enter key signal).

5.3 Software Interfaces

- Database: MySQL 8.x, accessed exclusively via JDBC (MySQL Connector/J version 8.0.x or later). All DDL scripts are provided as versioned migration files in the repository.
- Barcode library: ZXing Core version 3.5.x (zxing-core.jar and zxing-javase.jar), managed as Maven dependencies.
- Operating system interface: No Java Native Interface (JNI) is required; scanner input is treated as standard keyboard events.
- CSV export: Generated using Java's java.io.PrintWriter with UTF-8 encoding; no third-party CSV library is required.

5.4 Communication Interfaces

- All communication between gate client workstations and the MySQL database server shall use the MySQL wire protocol over TCP/IP (default port 3306).

- JDBC connection parameters (host, port, database name, username, password) shall be configurable through the Settings panel (FR-07) and persisted in an encrypted local properties file.
- For the production deployment, a VPN or dedicated VLAN is recommended by the IT Department to isolate database traffic; this is a deployment recommendation, not a hard system requirement for the development phase.

6. Requirements Traceability Matrix

The following matrix maps each functional and key non-functional requirement to the corresponding project objective (as defined in the Smart Gate Project Plan, §1.3) and to the relevant IEEE Std 830-1998 section.

Req. ID	Requirement Title	Project Objective	IEEE 830 §
FR-01	Card Scanning	Reduce gate passage time to < 10 s; automated identity verification	5.3.2
FR-02	Real-Time Activity Logging	Complete digital audit trail with ENTRY/EXIT, timestamp, gate ID	5.3.2
FR-03	Manual Fallback Entry	99% uptime with graceful fallback; continuity during scanner failures	5.3.2
FR-04	Campus Statistics Summary	Self-service reporting; real-time dashboard for administrators	5.3.2
FR-05	User Registration	Core functional requirement; user and card management	5.3.2
FR-06	Report Generation	Analytical reports within 60 s; data-driven security decisions	5.3.2
FR-07	Dashboard Navigation	Core functional requirement; administrator usability	5.2.1.2
FR-08	Authentication and RBAC	Security objectives NFR-05 to NFR-08	5.3.6.3
FR-09	Automated Database Backup	Data integrity and disaster recovery	5.3.6.1
FR-10	Audit Log Immutability	Tamper-proof audit trail; regulatory and institutional compliance	5.3.6.3
FR-11	Offline Caching & Sync	99% availability; no data loss during network interruptions	5.3.6.1
NFR-01	Scan Response Time < 2 s	Reduce gate passage time to under 10 s	5.3.3
NFR-05–08	Security Controls	Password security, SQL injection prevention, session management	5.3.6.3
NFR-09–11	Usability Metrics	90% task success; 15-minute training; SUS ≥ 70	5.2.3
NFR-12	99% System Uptime	High availability during university operating hours	5.3.6.2
NFR-15	MVC Architecture	Maintainability; future extensibility	5.3.6.4
NFR-17	3NF Database Schema	Data integrity; elimination of redundancy	5.3.4

Appendix A: Database Schema

The following schema defines all tables in the smart_gate_db MySQL database. The schema is normalised to Third Normal Form (3NF) with all foreign key constraints enforced at the database level.

Table	Column	Type / Constraint	Description
users	id	INT PK AUTO_INCREMENT	Internal surrogate key.
	id_number	VARCHAR(20) UNIQUE NOT NULL	University-issued student or staff ID.
	full_name	VARCHAR(100) NOT NULL	Display name used in activity logs.
	role	ENUM('STUDENT','EMPLOYEE','ADMIN') NOT NULL	Determines RBAC privileges.
	password_hash	VARCHAR(64) NOT NULL	Salted SHA-256 hash of the user's password.
	salt	VARCHAR(32) NOT NULL	Random salt prepended before hashing.
	email	VARCHAR(100)	Optional contact email.
cards	card_uid	VARCHAR(50) PK	UID decoded from the barcode card.
	user_id	INT NOT NULL FK → users(id)	One-to-one binding to a user record.
	issued_date	DATETIME DEFAULT CURRENT_TIMESTAMP	Date and time the card was registered.
gate_activity	activity_id	BIGINT PK AUTO_INCREMENT	Unique surrogate key for each gate event.
	user_id	INT FK → users(id)	The user who triggered the gate event.
	id_number	VARCHAR(20) NOT NULL	Denormalised for rapid log display without a join.
	action	ENUM('ENTRY','EXIT') NOT NULL	Direction of movement through the gate.

Table	Column	Type / Constraint	Description
	gate	VARCHAR(20) NOT NULL	Gate identifier (e.g., Gate A, Gate B).
	remarks	TEXT	NULL for automated scans; 'manual entry' for FR-03.
	created_at	DATETIME DEFAULT CURRENT_TIMESTAMP	Server-side timestamp; immutable after insert.
security_log	log_id	INT PK AUTO_INCREMENT	Unique key for each security event.
	event_type	VARCHAR(50) NOT NULL	E.g., 'FAILED_LOGIN', 'ACCESS_DENIED'.
	details	TEXT	Contextual description of the event.
	ip_address	VARCHAR(45)	IPv4 or IPv6 address of the client workstation.
	timestamp	DATETIME DEFAULT CURRENT_TIMESTAMP	Server-side time of the security event.

Appendix B: Acceptance Criteria Summary

Criterion	Target	Verification Method
Average gate passage time (scan to response)	≤ 2 seconds	Stopwatch timing with 50 participants; automated load test.
Gate events captured with complete audit attributes	100%	Post-UAT database audit query; zero missing fields.
Administrator report generation time ($\leq 10,000$ rows)	≤ 60 seconds	Timed execution on target server hardware.
System availability during operating hours (07:00–22:00)	$\geq 99\%$	Uptime monitoring log review during pilot deployment.
First-attempt task success rate (guards, students, staff)	$\geq 90\%$	Observed UAT sessions with structured test scripts.
Required training time for security guards	≤ 15 minutes	Timed training session during UAT; independent task trial.
System Usability Scale (SUS) score	$\geq 70 / 100$	SUS questionnaire administered post-UAT ($n \geq 10$).
Zero critical Checkstyle / SonarLint violations	0 violations	Static analysis run on final codebase at project sign-off.
Immutability of gate_activity records	0 modifications	Penetration test: direct UPDATE / DELETE attempts rejected.
Password storage compliance	0 plain-text passwords	Database inspection; log file review; network capture analysis.

Appendix C: Glossary

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability — properties guaranteeing reliable database transactions.
Cron Job	A time-based job scheduler in Unix/Linux operating systems, used here to automate daily database backups.
DAO	Data Access Object — a design pattern encapsulating all database interaction logic for a specific entity.
DDL	Data Definition Language — SQL statements that define database structure (CREATE, ALTER, DROP).
Foreign Key	A database constraint ensuring that a value in one table's column corresponds to a valid value in another table's primary key column.
InnoDB	The default MySQL storage engine, supporting ACID-compliant transactions and foreign key constraints.
Javadoc	Oracle's documentation comment standard for Java, used to generate HTML API documentation from source code.
JUnit 5	The fifth major version of the Java unit testing framework, used for automated test execution in the CI pipeline.
mysqldump	A command-line utility that produces a logical backup of a MySQL database as a SQL script file.
PreparedStatement	A precompiled SQL statement in JDBC that uses parameter placeholders, preventing SQL injection attacks.
Salted Hash	A hash computed from a password concatenated with a random value (salt), preventing rainbow-table attacks.
WCAG 2.1	Web Content Accessibility Guidelines 2.1 — international standard for accessible digital content, referenced here for UI colour contrast.